

UEBUNG_2

Übung 2 : Routing nach Attributen mit Parameter Context

Apache NiFi 2.8.0 · UI: <https://localhost:8443/nifi> · Login admin N=/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0 Fertiges Referenz-Lab auf der Canvas: “**Uebung 2 - Routing & Parameter**”

Lernziel: FlowFiles anhand ihrer **Attribute** in verschiedene Pfade lenken (RouteOnAttribute), inkl. **Catch-All (unmatched)**, und die Zielordner über einen **Parameter Context** (`#{...}`) zentral steuern statt sie hart in jeden Prozessor zu schreiben.

Flow:

```
GenerateFlowFile —> UpdateAttribute —> RouteOnAttribute —high—> PutFile
#{target_high}
    (Testdaten)          (priority,source) | —normal—> PutFile
#{target_normal}                                —unmatched—> PutFile
#{target_other}
```

Neues Konzept: Parameter Context (`#{...}` vs. `${...}`)

Syntax	Was	Wann ausgewertet
<code>\${attribut}</code>	Expression Language : liest FlowFile-Attribute/ Funktionen	zur Laufzeit pro FlowFile
<code>#{parameter}</code>	Parameter aus dem Parameter Context	beim Konfigurieren/Start

`#{...}` ist wie eine Konfigurationsvariable: einmal zentral gesetzt, in vielen Prozessoren verwendbar. Ideal für Pfade, Hosts, Schwellenwerte : und für dev/prod-Umschaltung.

Schritt 0 : Parameter Context anlegen und binden

1. Hamburger-Menü  → **Parameter Contexts** → +.
2. Name: Uebung2-Params. Tab **Parameters** → + für jeden Eintrag:
 - `target_high = $N/data/output/uebung2_high`
 - `target_normal = $N/data/output/uebung2_normal`
 - `target_other = $N/data/output/uebung2_other` (Pfade vollständig, z.B. `/Users/grigo/.../nifi-2.8.0/data/output/uebung2_high`.)
3. **Apply.**

4. Die **Process Group** an den Context binden: ⚙ der PG → **General** → **Process Group Parameter Context** = `Uebung2-Params` → Apply.

Erst nach dem Binden kennt die PG die `#{...}`-Parameter.

Schritt 1 : Testdaten vorbereiten (GenerateFlowFile)

Doppelklick → **Properties**: - **Custom Text** = `Datensatz erzeugt ${now():format('HH:mm:ss')}`
- **Data Format** = `Text`, **Batch Size** = `1`

Tab **Scheduling**: **Run Schedule** = `30 sec`

Alternativ echte Testdatei via `GetFile` : für die Routing-Demo reicht `GenerateFlowFile`.

Schritt 2 : UpdateAttribute setzt priority und source

Doppelklick → **Properties** → mit **+**: - `priority = high` (zum Testen später auf `normal` / `low` ändern) - `source = erp` (alternativ `crm`)

Diese Attribute sind die **Routing-Kriterien**. In der echten Welt kämen sie z.B. aus dem Dateinamen, HTTP-Header oder einem vorgelagerten `ExtractText`.

Variante zum Testen (morgen ausprobieren): priority automatisch zufällig

Statt `priority` von Hand zwischen `high`/`normal` umzustellen, kann `UpdateAttribute` den Wert pro FlowFile selbst würfeln. Dann fließen automatisch beide Sorten und man sieht das Routing live:

```
priority = ${random():mod(2):equals(0):ifElse('high','normal')}
```

So kombiniert: `random()` erzeugt eine Zufallszahl, `mod(2)` macht daraus 0 oder 1, `equals(0)` ergibt `true/false`, `ifElse('high','normal')` wählt entsprechend den Wert. Für den `unmatched`-Pfad (z.B. `low`) weiterhin manuell einen anderen Wert setzen, oder einen dritten Zweig mit `mod(3)` bauen.

Schritt 3 : RouteOnAttribute mit Regeln + unmatched

Doppelklick → **Properties**: - **Routing Strategy** = `Route to Property name` - Mit **+** zwei dynamische Properties (Name wird zur Relationship): - `high = ${priority:equals('high')}` - `normal = ${priority:equals('normal')}`

Relationships (Tab Relationships): es gibt jetzt `high`, `normal` und `unmatched`. - `unmatched` ist der **Catch-All**: jede FlowFile, die auf KEINE Regel passt (z.B. `priority=low`).

Die EL-Ausdrücke geben `true/false` zurück. `Route to Property name` schickt die FlowFile über die Relationship, deren Ausdruck `true` ist. Trifft keine zu → `unmatched`.

Schritt 4 : Drei PutFile, parameter-gesteuert

Prozessor	Directory (Property)
PutFile HIGH	<code>#{target_high}</code>
PutFile NORMAL	<code>#{target_normal}</code>
PutFile OTHER	<code>#{target_other}</code>

Je: Create Missing Directories = `true`, Conflict Resolution = `replace`.

Statt drei fester Pfade tippt man `#{target_high}` usw. : NiFi setzt den Wert aus dem Parameter Context ein. Will man später alle Ausgaben umlenken, ändert man nur den Parameter, nicht 3 Prozessoren.

Schritt 5 : Verdrahtung

Von	Relationship	Nach
GenerateFlowFile	success	UpdateAttribute
UpdateAttribute	success	RouteOnAttribute
RouteOnAttribute	high	PutFile HIGH
RouteOnAttribute	normal	PutFile NORMAL
RouteOnAttribute	unmatched	PutFile OTHER
alle PutFile	success, failure	terminieren

Schritt 6 : Regeln dokumentieren

Regel (Relationship)	Bedingung	Ziel-Parameter	Beispiel-priority
<code>high</code>	<code>\${priority:equals('high')}</code>	<code>#{target_high}</code>	<code>high</code>
<code>normal</code>	<code>\${priority:equals('normal')}</code>	<code>#{target_normal}</code>	<code>normal</code>
<code>unmatched</code>	(keine Regel trifft)	<code>#{target_other}</code>	<code>low</code> , <code>leer</code> , <code>urgent</code> ...

Schritt 7 : Testfälle pro Priorität durchspielen

Pro Testfall: UpdateAttribute `priority` ändern (Apply), Flow kurz starten, Ordner prüfen, stoppen.

```
H=$N/data/output/uebung2_high; M=$N/data/output/uebung2_normal; O=$N/data/output/uebung2_other
ls $H $M $O
```

Erwartung (im Live-Test bestätigt):

priority gesetzt	Datei landet in
high	uebung2_high ✓
normal	uebung2_normal ✓
low	uebung2_other (unmatched) ✓

Tipp Schulung: zusätzlich `source=erp` vs. `crm` variieren und eine zweite Regel ergänzen, z.B. `crm_high = ${priority>equals('high'):and(${source>equals('crm')})}` : zeigt UND-Verknüpfung.

Reset

```
find $N/data/output/uebung2_high $N/data/output/uebung2_normal $N/data/output/
uebung2_other -type f -delete
# Process Group stoppen; priority wieder auf 'high' stellen
```

Häufige Stolpersteine

- `#{target_high}` **wird wörtlich verwendet / PutFile invalid** → Parameter Context nicht an die PG gebunden, oder Parametername vertippt.
- **Alles geht nach unmatched** → EL-Ausdruck falsch (z.B. `${priority>equals(high)}` ohne Anführungszeichen) oder Attribut `priority` nicht gesetzt.
- **Keine high/normal-Relationship sichtbar** → Routing Strategy muss `Route to Property name` sein; die Relationships entstehen erst durch Anlegen der dynamischen Properties.
- **Output flutet** → `GenerateFlowFile` nach dem Test stoppen (läuft alle 30s).